EhBASIC for the 6502 SBC — Manual

Nov 1, 2026 · @dylan

This is EhBASIC 2.22 as built into your SBC's ROM: the standard language, plus commands for the USB drive (CH376), the I2C bus and LCD, the gamepad and the AY-3-8910 sound card. Keywords can be typed in upper or lower case.

Starting up

At power-up or reset the ROM sets up the hardware (serial port, display card, VIA, gamepad, I2C, sound card silenced) and asks:

`[C]old start, [W]arm start, [M]onitor?`

Key	What it does
C	Cold start: BASIC starts empty and asks `Memory size ?`
W	Warm start: back into BASIC with the program and variables kept (after a crash or a trip to the monitor)
M	The Woz monitor (see The monitor)

Memory size ? Press Enter to give BASIC all of RAM ($0500–$7EFF, about 31 KB). Type a smaller number (an address) to keep the RAM above it free for machine code or LOADRAW data.

Input comes from the keyboard/display card, the serial port and the gamepad, all into the same buffer. Ctrl-C stops a running program (`Break in line n`); CONT carries on.

Typing programs: a line starting with a number is stored; anything else runs at once. Type the number alone to delete a line. Several statements go on one line separated by `:` . `?` is short for PRINT.

Quick reference

Every keyword in this ROM. "Added" marks the ones built for your board; the rest are standard EhBASIC 2.22.

Group	Keywords	Where
Program	`RUN LIST NEW CLEAR CONT END STOP REM`	Standard
Flow	`GOTO GOSUB RETURN ON..GOTO/GOSUB IF..THEN..ELSE FOR..TO..STEP NEXT DO LOOP WHILE/UNTIL`	Standard
Data	`LET DIM READ DATA RESTORE INPUT GET SWAP INC DEC DEF FN`	Standard
Output	`PRINT (?) TAB( SPC( WIDTH NULL`	Standard
Memory	`PEEK( POKE DEEK( DOKE BITSET BITCLR BITTST( WAIT CALL USR( VARPTR( SADD(`	Standard
Interrupts	`ON IRQ ON NMI IRQ ON/OFF/CLEAR NMI ON/OFF/CLEAR RETIRQ RETNMI`	Standard
Maths	`ABS( INT( SGN( SQR( RND( LOG( EXP( SIN( COS( TAN( ATN( MAX( MIN( PI TWOPI FRE( POS(`	Standard
Strings	`LEN( LEFT$( RIGHT$( MID$( STR$( VAL( ASC( CHR$( UCASE$( LCASE$( HEX$( BIN$(`	Standard
Operators	`+ - * / ^ AND OR EOR NOT << >> = < > <= >= <>`	Standard
USB drive	`LOAD SAVE LOADRAW SAVERAW LOADARR SAVEARR`	Added
I2C / LCD	`I2CINIT I2CWRITE I2CREAD( LCDINIT LCDPRINT LCDPOS LCDCLS LCDCMD`	Added
Sound	`SOUND NOTE NOISE ENVELOPE QUIET AYREG AYREAD(`	Added
Gamepad	`PADKEY`	Added

Standard statements

Variable names: the first two characters count (`AB` and `ABC` are the same variable). `A` is a number, `A$` a string; arrays are `A(n)` or `A$(n)`. Numbers are floating point; integer arguments run 0–65535 (or -32768–65535 where noted). Hex constants are written `$1F`, binary `%1010`.

Statement	Syntax	What it does
RUN	`RUN [line]`	Clears variables and runs from the start or the line given
LIST	`LIST [a][-[b]]`	Lists the program: `LIST 100-200`, `LIST -50`, `LIST 300-`
NEW	`NEW`	Deletes the program and variables
CLEAR	`CLEAR`	Clears variables, arrays and strings; keeps the program
CONT	`CONT`	Carries on after STOP, END or Ctrl-C
END / STOP	`END` `STOP`	Ends the program; STOP prints `Break in line n`
REM	`REM text`	A comment, to the end of the line
LET	`[LET] var = expr`	Assignment; LET is optional
DIM	`DIM A(10), B$(5,5)`	Arrays (index 0 to n); undimensioned arrays default to 10
GOTO / GOSUB	`GOTO line` `GOSUB line` `RETURN`	Jump; call a subroutine and come back
ON	`ON n GOTO l1,l2,..` `ON n GOSUB ..`	Goes to the nth line; n out of range: carries on
IF	`IF cond THEN line` or `IF cond THEN stmts [ELSE stmts]`	`IF A>3 THEN 100` `IF X THEN PRINT 1 ELSE PRINT 2`
FOR / NEXT	`FOR I = a TO b [STEP s]` .. `NEXT [I[,J]]`	Counted loop
DO / LOOP	`DO` .. `LOOP`, `LOOP WHILE cond` or `LOOP UNTIL cond`	Loop; plain LOOP repeats for ever

Statement	Syntax	What it does
READ / DATA	`READ A, B$` `DATA 1, "TEXT"` `RESTORE`	Reads the DATA lines in order; RESTORE starts again
INPUT	`INPUT ["prompt";] A, B$`	Waits for a typed line
GET	`GET A$` or `GET A`	Reads one key without waiting: empty string / 0 if none
PRINT	`PRINT expr ; expr , expr ? ..`	`;` joins, `,` goes to the next tab stop, a trailing `;` stops the new line
TAB(/ SPC(	`PRINT TAB(10);"X"; SPC(3);"Y"`	Move to column n / print n spaces (in PRINT only)
WIDTH	`WIDTH n[,t]` `WIDTH ,t`	Line width (16–255, 0 = no limit) and tab stop size
NULL	`NULL n`	n NUL characters after each new line (for slow terminals)
SWAP	`SWAP A, B`	Exchanges two variables of the same type
INC / DEC	`INC A[,B..]` `DEC A`	Adds or takes 1
DEF FN	`DEF FNA(X) = X*X+1` then `FNA(3)`	One-line function of one variable
POKE / DOKE	`POKE addr, byte` `DOKE addr, word`	Writes a byte / a 16-bit word (low byte first)
BITSET / BITCLR	`BITSET addr, bit` `BITCLR addr, bit`	Sets / clears bit 0–7 of a byte in memory
WAIT	`WAIT addr, and [,eor]`	Waits until `(PEEK(addr) EOR eor) AND and` is not 0
CALL	`CALL addr`	Runs machine code at addr; it ends with RTS

The interrupt statements (ON IRQ, IRQ ON and the rest) are under Interrupts and machine code.

Functions and operators

Function	Returns
`ABS(x)` `INT(x)` `SGN(x)`	Absolute value; rounded down; -1, 0 or 1
`SQR(x)` `LOG(x)` `EXP(x)`	Square root; natural log; e to the power x
`SIN(x)` `COS(x)` `TAN(x)` `ATN(x)`	Trigonometry, in radians
`PI` `TWOPI`	3.14159.. and 6.28318..
`RND(x)`	Random 0 to <1: x>0 next number, x=0 the last one again, x<0 seeds with x
`MAX(a,b,..)` `MIN(a,b,..)`	Largest / smallest of the list
`FRE(0)`	Free bytes of BASIC memory
`POS(0)`	The cursor's column
`PEEK(a)` `DEEK(a)`	The byte / 16-bit word at address a
`BITTST(a,b)`	-1 if bit b of the byte at a is set, else 0
`VARPTR(var)` `SADD(s$)`	Address of a variable / of a string's characters
`USR(x)`	Calls machine code through the vector at $0A (see Interrupts and machine code)
`LEN(s$)` `ASC(s$)` `VAL(s$)`	Length; code of the first character; the number in the string
`STR$(x)` `CHR$(n)`	Number as a string; one character with code n
`LEFT$(s$,n)` `RIGHT$(s$,n)`	The first / last n characters
`MID$(s$,p[,n])`	n characters from position p (1 = first); to the end without n
`UCASE$(s$)` `LCASE$(s$)`	Upper / lower case
`HEX$(x[,n])` `BIN$(x[,n])`	Hex (up to 6 digits) / binary (up to 24) text; n = minimum digits
`I2CREAD(addr)` `AYREAD(reg)`	Added: see I2C and Sound

Operators, from first to last: `^`; unary `-`; `* /`; `+ -`; `<< >>` (shifts); comparisons `= <> < > <= >=` (true = -1, false = 0); `NOT`; `AND`; `OR` and `EOR`. AND, OR and EOR work bit by bit on 16-bit integers, so `PEEK(a) AND 4` tests a bit. `+` also joins strings.

USB drive

Files live on the USB stick in the CH376 card. Names are 8.3, turned into upper case, and may be any string expression. BASIC's file commands work from the root directory; `"DIR/NAME"` reaches a sub-directory.

Command	Example	What it does
`LOAD "name"`	`LOAD "STARTREK"`	Loads `/NAME.BAS` (.BAS added when no extension); the old program is replaced
`SAVE "name"`	`SAVE "GAMES/MAZE"`	Saves the program as `NAME.BAS` (tokenised)
`LOAD "$"`	`LOAD "$"`	Lists the root directory; the program in memory is not touched
`LOAD "$pattern"`	`LOAD "$*.BAS"` `LOAD "$GAME*"`	Lists only the matching files
`SAVERAW "name", start, end`	`SAVERAW "SCREEN", 32000, 32512`	Saves the bytes start to end-1 as `NAME.DAT`
`LOADRAW "name" [, start]`	`LOADRAW "SCREEN", 32000`	Loads a .DAT file to start; without start, to the first byte above BASIC's memory
`SAVEARR "name", A()`	`SAVEARR "SCORES", S()`	Saves a numeric array as `NAME.ARR`
`LOADARR "name", A()`	`DIM S(9) : LOADARR "SCORES", S()`	Loads it back; DIM the array first, with the same sizes

LOADRAW is fenced in. It only loads above BASIC's memory (the RAM you kept free at `Memory size ?`) and below $7F00, so it can't overwrite the program, variables or system RAM. To make room, answer `Memory size ?` with, say, `30000`. String arrays can't be saved with SAVEARR (`Type mismatch`).

Disk messages: `?FILE NOT FOUND`, `?DISK FULL`, `?FILE TOO LARGE`, `?BAD FILE`, `?NO DISK`, and `?DISK ERROR $nn (CMD $nn, PORT $nn)` with the CH376's status codes ($FF = the chip never answered).

I2C bus and LCD

The PCF8584 bridge and the LCD set themselves up the first time they're used, so I2CINIT and LCDINIT are optional. Addresses are the 7-bit datasheet ones (0–127, not shifted). A device that doesn't answer a write gives `I2C Error`.

Command	Example	What it does
`I2CINIT`	`I2CINIT`	Resets the PCF8584
`I2CWRITE addr [, b1, b2 ..]`	`I2CWRITE $20, 0, 255`	Writes up to 32 bytes; with no bytes it just checks the device is there
`x = I2CREAD(addr)`	`X = I2CREAD($48)`	Reads one byte; -1 if nothing answers (no error)
`LCDINIT [addr [, type]]`	`LCDINIT $27, 1`	Picks the LCD: type 0 = MCP23008 backpack (default, address $20), 1 = PCF8574 backpack
`LCDPRINT s$`	`LCDPRINT "TEMP " + STR$(T)`	Writes text at the cursor
`LCDPOS col, row`	`LCDPOS 0, 1`	Moves the cursor (col 0–39, row 0–3)
`LCDCLS`	`LCDCLS`	Clears the display
`LCDCMD n`	`LCDCMD 12`	Sends an HD44780 command: 12 = cursor off, 15 = blinking cursor

Your LCD has a PCF8574 backpack, so start programs with `LCDINIT addr, 1` (usually $27 or $3F; plain `LCDINIT` goes back to the MCP23008 at $20). A bus scan:

```
10 FOR A = 8 TO 119
20 IF I2CREAD(A) >= 0 THEN PRINT HEX$(A)
30 NEXT
```

Sound card

The AY-3-8910 has three channels: 0, 1 and 2 (A, B, C). Volume is 0–15, or 16 to follow the envelope. Reset leaves the card silent.

Command	Example	What it does
`SOUND ch, freq, vol [, time]`	`SOUND 0, 440, 12`	A tone in Hz (31–65535)
`NOTE ch, n, vol [, time]`	`NOTE 1, 45, 10, 30`	A tone by note number (below)
`NOISE ch, period, vol [, time]`	`NOISE 2, 5, 9, 6`	Noise instead of a tone; period 0–31, 0 = highest hiss
`ENVELOPE shape, period`	`ENVELOPE 0, 3000`	The shared envelope used by volume 16
`QUIET`	`QUIET`	All channels off
`AYREG reg, value`	`AYREG 7, 56`	Writes any chip register 0–15 directly
`x = AYREAD(reg)`	`PRINT AYREAD(14)`	Reads a register back; 14 and 15 are I/O ports A and B. Needs the rev 2 GAL (the old card gives 255)

time is in 1/60 seconds. A timed note switches itself off while the program carries on. A timed note on a channel first waits for that channel's previous timed note to end, so a tune is one statement per note. Each channel waits on its own, so channels can play together. Ctrl-C stops a wait. Leave time out (or 0) and the note plays until changed.

```
10 REM A SCALE, A QUARTER SECOND A NOTE
20 FOR I = 1 TO 8 : READ N : NOTE 0, N, 12, 15 : NEXT
30 DATA 36,38,40,41,43,45,47,48
```

Note numbers run 0 (C1) to 95 (B8): n = (octave − 1) × 12 + the step below. Middle C (C4) is 36; A4 (440 Hz) is 45.

C	C#	D	D#	E	F	F#	G	G#	A	A#	B
0	1	2	3	4	5	6	7	8	9	10	11

Envelope shapes: 0 = one fade out, 13 = fade in and hold, 8 = repeating saw down, 12 = saw up, 10 or 14 = triangle. Each ramp lasts period × 128 µs, so `ENVELOPE 0, 3000` fades over about 0.4 s. Use it with volume 16: `ENVELOPE 0, 3000 : NOTE 0, 36, 16`.

Good to know: times need the VIA's 60 Hz timer interrupt (always on unless a program turned it off). If a long timed note is still playing when you start a monitor program like YMPLAY, type `QUIET` first. The PC-side tokenizer, if you use one, doesn't know these keywords yet.

Gamepad

The NES-style pad on the VIA (PA0 data, PA1 latch, PA2 clock) is read 60 times a second. Each press types a character into the input buffer, so GET and INPUT see it like a key, and a held button repeats.

Button	Number	Default key
A	0	Space
B	1	none
Select	2	none
Start	3	none
Up	4	W
Down	5	S
Left	6	A
Right	7	D

`PADKEY button, code` sets a button's character: `PADKEY 1, 13` makes B the Enter key, `PADKEY 3, ASC("P")` makes Start type P. Code 0 switches a button off. Plain `PADKEY` puts the defaults back. To read the buttons held right now, machine code can call the jump table's PADSTATE at $FE12 (bit n = button n).

Interrupts and machine code

BASIC interrupt handlers. `ON IRQ line` names a handler and turns it on; the handler ends with `RETIRQ`. The handler runs at the end of the statement that was running when the interrupt came, and must clear the interrupt's source itself. `IRQ OFF` holds interrupts back, `IRQ ON` lets them through again, `IRQ CLEAR` forgets one that's waiting. `ON NMI line`, `NMI ON/OFF/CLEAR` and `RETNMI` work the same for NMI.

The ROM keeps IRQ3 (serial), IRQ5 (keyboard/display) and the VIA's timer 1 (gamepad and sound timing) for itself. IRQ0, IRQ1, IRQ4, IRQ6, IRQ7 and any other VIA interrupt are passed to ON IRQ.

Machine code. `CALL addr` runs a routine that ends with RTS. `USR(x)` jumps through the vector at $0B/$0C (a JMP at $0A): set it with `DOKE 11, addr`; the routine finds x in the floating point accumulator. Keep code above BASIC's memory (answer `Memory size ?` lower), load it with `LOADRAW`, and use the jump table rather than ROM addresses, which move when the ROM is rebuilt.

Address	Name in sbc_api.inc	What it does
$FE00	CHRIN	Carry set + A = a character, or carry clear if none waiting
$FE03	CHROUT	Prints the character in A (A, X, Y kept)
$FE06	MENU	Back to the boot menu
$FE09	COLD	EhBASIC cold start
$FE0C	WARM	EhBASIC warm start
$FE0F	MONITOR	The monitor prompt
$FE12	PADSTATE	A = gamepad buttons held (bit n = button n)
$FE15	LOAD	Loads a file into RAM silently (A/Y = parameter block)
$FE18	CWD	A/Y = address of the monitor's current folder name

`CALL 65280` ($FF00) goes from BASIC to the monitor.

The monitor

The Woz monitor starts from the boot menu (M) or from BASIC with `CALL 65280`. Everything is in hex; lower case is fine. Ctrl-C at its prompt goes back to the boot menu.

Command	Example	What it does
`XXXX`	`0500`	Shows the byte at an address
`XXXX.YYYY`	`0500.05FF`	Shows a range
`XXXX: AA BB ..`	`0500: A9 41 60`	Stores bytes from that address
`XXXXR`	`0500R`	Runs the program at XXXX
`?`	`?`	Lists the commands
`L`	`L`	Lists the current folder on the USB drive
`L DIR`	`L GAMES` `L ..` `L /`	Changes folder: into GAMES, up one, back to the top (`L A/B` goes several deep)
`L NAME [XXXX]`	`L INVADERS 0500`	Loads `NAME.BIN` to XXXX ($0500–$7EFF); then `R` runs it

Without XXXX, `L NAME` loads to the last address typed. The folder goes back to the top at every reset.

Memory map and error messages

Address	Used for
`$0000-$01FF`	Zero page and stack
`$0200-$04FF`	Monitor line buffer, CH376 and BIOS variables, BASIC's input buffer, the keyboard/serial buffer
`$0500-$7EFF`	BASIC program, variables and strings (the top part is yours if you answer `Memory size ?` lower)
`$7F00-$7F0F`	65C22 VIA (gamepad on port A, 60 Hz timer)
`$7F10-$7F11`	PCF8584 I2C

Address	Used for
`$7F14-$7F17`	Serial ACIA
`$7F18-$7F1B`	Keyboard/display card
`$7F20, $7F21`	CH376 data, command
`$7F22, $7F23`	AY-3-8910 register, data
`$7FFF`	Interrupt register
`$8000-$FDFF`	EhBASIC and the BIOS
`$FE00-$FEFF`	Jump table
`$FF00-$FFFF`	Woz monitor and vectors

Errors print as `Name Error` (with `in line n` when a program was running):

Message	Usual cause
Syntax	Mistyped statement, or a statement keyword (like SOUND) used in an expression
Function call	An argument out of range: a channel above 2, volume above 16, a bad address
Type mismatch	A string where a number was wanted, or the other way round
Out of memory	Program, arrays or GOSUB/FOR nesting too big
Overflow / Divide by zero	Number too large; division by 0
Array bounds / Double dimension	Index past the DIM size; the same array DIMed twice
Undefined statement / Undefined function	GOTO a missing line; FN not DEFined
NEXT without FOR / RETURN without GOSUB / LOOP without DO	Loop or subroutine mismatch
Out of DATA	READ ran past the last DATA

Message	Usual cause
String too long / String too complex	Over 255 characters; too many strings in one expression
Illegal direct / Can't continue	A statement that only works in a program; CONT after an edit or error
I2C	A device didn't answer (I2CWRITE, LCD commands)

Disk errors are listed under USB drive. `Break in line n` means Ctrl-C or STOP; type CONT to carry on.